

天気情報エージェントとMCP

受講者用ワークブック：操作手順と演習

著者：Rajkumar Rajagobalan

ドラフト v1.0 日本語版 | 2026年9月10日 | Google Colabで学ぶ2時間ワークショップ

同じ天気の質問を3通りの方法で処理し、それぞれの仕組みと役割分担を理解します。

このワークブックでは、通常の天気API呼び出しから始め、モデルが関数ツールを使うエージェントを実行します。最後に、同じ天気取得機能をModel Context Protocol (MCP) 経由で利用します。

演習を開く

[Google Colabでノートブックを開く](#)

[教材リポジトリ：nuvear/Agents-Basics](#)

ワークブックとノートブックを並べて開いてください。実行するコードはノートブックに用意されています。本書のコード例は、該当する処理を見つけたり、再実行したりする際に使います。必ず準備用セルから順番に実行してください。英語のセル名や出力は、ノートブックと照合できるよう原表記を残しています。

この講座の到達目標

- モデルによるツール実行の要求と、実際の実行を区別できる。
- 提供元またはサンプルから得た天気データが、エージェントを通る経路を追える。
- MCPのホスト、クライアント、サーバーを識別できる。
- ツールの検出、引数、実行、返されたデータを確認できる。
- サンプルデータ、台本に沿ったモデル応答の代替処理、実際のモデル推論の違いを説明できる。

受講者記入欄

氏名： _____

ペア／グループ： _____

受講日： _____

演習の基本：もっともらしい回答だけでは、処理が実行された証拠にはなりません。実行ログと返されたデータを確認しましょう。

1. ワークショップの進め方

受講前の準備：第2章と第3章を済ませておいてください。必要なのは、ブラウザ、インターネット接続、Googleアカウントです。標準のホスト型Colab Pythonランタイムを使います。GPUや、手元のPCへのPython・LM Studioのインストールは不要です。

経過時間	内容	該当章
0-10分	天気の質問と各要素の役割	1
10-25分	APIを直接呼び出す演習	4
25-50分	関数呼び出し型エージェントの演習	5-6
50-55分	休憩	-
55-70分	MCPの役割と責任範囲	7
70-100分	MCPのツール検出とエージェントの演習	8
100-113分	ペア課題と異常系テスト	9
113-120分	理解度確認	11

実行方法を確認する

実行方法	モデルの動作	天気データの扱い
OpenAI+サンプル	実際に推論する。API利用が発生する	合成データ。SerpApiキーは不要
キーなしの代替モード	東京を指定する固定の要求。モデル推論は行わない	合成データ。ローカルのツールは実際に実行する
任意のライブデモ	OpenAI設定済みなら実際に推論する	SerpApiにリクエストする。別の天気サービス用キーが必要

代替モードでもMCP接続は実際に行われます。ツールサーバーはColabランタイム内で動作します。模擬処理なのはモデルとのやり取りと天気の値です。

ウォームアップ：実行前に予想する

ユーザーが「東京の現在の気温は？」と質問したとします。

- 1. モデルが学習時に得ている可能性のある情報は何でしょうか。
- 2. 今、外部の情報源から取得しなければならない情報は何でしょうか。
- 3. 何を確認すれば、天気ツールが実際に動いたと判断できるでしょうか。

予想： _____

APIは、ソフトウェアから利用するためのインターフェースです。本講座でいうエージェントは、モデル、ツール、会話の状態、実行を制御するループを組み合わせたアプリケーションです。MCPは、ホスト内のクライアントとツールサーバーの間の通信を標準化します。

2. Colabと演習ファイルを準備する

受講前 | ノートブック：Preparation A、Preparation B

手順1. 公開ノートブックを開く

表紙のColabリンクを開くか、リポジトリの Open in Colab をクリックします。必要に応じてGoogleアカウントにログインしてください。GitHub由来のノートブックについて警告が表示された場合は、リポジトリが nuvear/Agents-Basics であることを確認してから進みます。

手順2. 自分用のコピーを保存する

File > Save a copy in Drive（ファイルメニューからドライブにコピーを保存）を選びます。

MCP_Weather_YourName などの名前を付け、このコピーで作業してください。メモや変更を、公開教材とは分けて保存できます。

手順3. 標準ランタイムに接続する

Connect（接続）をクリックします。ランタイムの種類を選ぶ場合はPythonを使い、ハードウェアアクセラレータを None（なし）に設定します。接続が完了するまで待ってください。ホスト型ランタイムは、Colab セッション用に割り当てられる実行用コンピューターです。

手順4. セルの実行方法を確認する

コードセル横の三角形の実行ボタンをクリックするか、セルを選択して Shift+Enter を押します。次へ進む前に出力を読んでください。準備はA、B、C、Dの順に実行します。先行セルの定義を使うセルは、その定義が未実行だとエラーになります。

手順5. Preparation Aを実行する

Preparation A - create the lab files を見つけ、折りたたまれている準備用セルを実行します。同梱のスク립トとサンプルJSONがランタイム内に書き込まれます。

次の確認メッセージが表示されます。

```
Workshop files ready: /content/mcp_weather_2hour
```

Colabのファイルパネルで、このフォルダーを確認してください。この準備処理ではZIPの展開もGoogle Driveのマウントも行いません。同梱ソースを確認したい場合は、準備用セルを展開します。

手順6. Preparation Bを実行する

依存パッケージをインストールするセルを実行し、次のメッセージが出るまで待ちます。

```
Dependencies installed in the isolated lab environment.
```

演習専用の独立したPython環境が作成されます。インストールに失敗したら、そこで止めて講師に相談してください。後続セルを実行しても、欠けている依存パッケージは修復されません。

準備の記録： ファイル作成済み [] 依存パッケージ導入済み []

ランタイム内のファイルは一時的なものです。ランタイムが削除・再作成されたら、準備をやり直します。ノートブックを保存し、残したいファイルはダウンロードしてください。

3. モデルを設定し、動作準備を確認する

受講前 | ノートブック：Preparation C、Preparation D

手順7. OpenAIキーがある場合は登録する

Colabサイドバーの鍵アイコンから Secrets（シークレット）を開きます。名前を正確に OPENAI_API_KEY として追加し、値にキーを入力して、ノートブックからのアクセスを有効にします。キーをコードセル、ワークブックの回答欄、ノートブックの出力に貼り付けないでください。

キーがなくても先へ進めます。ノートブックは台本に沿った代替モードを選択します。キーが登録されていても、利用枠が残っていることや、選択したモデルを使えることまでは確認できません。

手順8. ライブ天気取得は任意のままにする

基本演習では SERPAPI_KEY を設定しません。任意のライブデモには、OpenAI とは別の SerpApi 認証情報が必要です。OpenAI キーで天気サービスに認証することはできません。

手順9. Preparation C を実行する

Colab Secrets を使う場合は USE_MASKED_PROMPT = False のまま、認証情報のセルを実行します。キーの値は表示せずに読み込み、選択したモードを表示します。

次のいずれかが表示されます。

```
OpenAI inference + sample weather  
Scripted model fallback + sample weather
```

Secrets が使えず、講師から伏せ字入力を使うよう案内された場合は、USE_MASKED_PROMPT = True に変更してセルを再実行し、入力欄にキーを入力します。ソースコード内の文字列を実際のキーに置き換えないでください。

手順10. Preparation D を実行する

run_lab() と model_options() を定義した後、サンプルデータを使って実際の MCP 接続を確認します。次の出力を探してください。

```
[MCP tools/list]  
[MCP tools/call] get_current_weather
```

結果には mock: true、または同等の JSON 表記が含まれます。ここで確認するのは Colab 内のローカル MCP の動作です。OpenAI の推論やライブ天気取得のテストではありません。

手順11. 開始時の状態を記録する

モード：OpenAI [] 台本に沿った代替モード []

MCP ツール検出に成功 [] サンプルのツール結果を取得 []

後からキーを追加した場合は、エージェント演習を再実行する前に Preparation C を再実行します。モデルは講師が事前確認したものを使ってください。ノートブックの既定値は gpt-4.1-mini です。

4. 演習1：通常のAPI呼び出しを確認する

10-25分 | ノートブック：Lab 1 - Direct API

手順12. 実行結果を予想する

この処理にモデルは関与しません。サンプル実行ではリクエストのパラメーターを組み立てますが、ネットワークへ送信せず、同梱の応答データを読み込みます。気温と単位がどこにあるか予想してください。

手順13. Lab 1の最初のセルを実行する

```
run_lab("01_direct_serpapi_api.py", "--sample", "--show-raw")
```

エンドポイントとクエリパラメーターを読み、未加工の応答にある `answer_box` を探します。続いて、形式を統一した結果から `temperature`、`location`、`observation_label`、`sources`、`mock` を確認してください。

表示されるAPIエンドポイントは `https://serpapi.com/search.json` です。サンプルの気温は摂氏21度です。これは教材用データであり、東京の現在の天気ではありません。

手順14. 確認したデータを記録する

項目	確認した内容
エンドポイント／クエリ（認証情報を除く）	_____
返された場所	_____
気温と単位	_____
観測情報のラベル	_____
情報源とサンプルを示す印	_____

手順15. 入力を1つ変える：単位

次のセルを実行する前に、華氏では何度になるか予想します。

```
run_lab("01_direct_serpapi_api.py", "--sample", "--units", "fahrenheit")
```

予想：_____ 結果：_____

確認：摂氏21度は華氏69.8度です。固定のテスト用データ（フィクスチャ）は変わりません。サンプルモードで都市の引数を変更しても、新しい天気データは取得されません。

手順16. 外部通信を行う箇所を見つける

ファイルパネルから `serpapi_weather.py` を開き、`perform_serpapi_search` を探します。ライブモードでは、この関数が提供元へリクエストを送信します。この演習では編集しません。

説明してみましょう：AIモデルがなくても、このプログラムがデータの取得と形式の統一を行えるのはなぜでしょうか。

5. 演習2：関数呼び出し型エージェントを実行する

25-50分の前半 | ノートブック：Lab 2 - Function-calling agent

手順17. 役割分担を確認する

アプリケーションが質問とツールのスキーマをモデルに送ります。モデルは引数を指定して関数の実行を要求します。アプリケーションはその引数を検証し、関数を実行して、結果をモデルに返します。

手順18. エージェントのセルを実行する

```
run_lab("02_custom_tool_agent.py", *model_options())
```

OpenAIを設定済みなら、合成の天気データを使いながら、モデルへのリクエストは実際に行います。キーがなければ東京を指定する固定の要求を使い、モデルによる最終回答の生成は行いません。どちらの経路を使ったかは実行ログで確認できます。

手順19. 実行の証拠を追う

出力から次の段階を探します。要求を示す行だけでは、実行された証拠にはなりません。

ログの段階	分かること
[MODEL REQUEST]	ホストがモデルとの1回のやり取りを開始した
[TOOL REQUEST]	モデルまたは代替処理が、名前を指定してツールを要求した
[VALIDATED ARGUMENTS]	ホストが引数を検証し、受け入れた
[EXECUTION]	Pythonが天気関数の実行を呼び出した
[TOOL RESULT]	関数からデータまたはエラーが返った
FINAL ANSWER	モデルの回答、または代替処理と明示された完了メッセージ

手順20. 結果と回答を照合する

要求された都市、国コード、単位を確認します。ツール結果の mock: true を確認してください。実際のモデルが回答を生成した場合は、サンプルデータであることが示されているはずです。回答中の数値とツール結果を照合します。

要求されたツール：_____

引数：_____

結果／回答内のサンプル表記：_____

手順21. 最初の呼び出しが必須である理由を説明する

OpenAI使用时、model_options() は --force-tool を指定します。そのため、ホストが最初のターンでツール使用を必須にし、モデルが引数を決めます。この実行から、モデルが自発的にツール使用を判断したとはいえません。

ツールが実行されなければ、ホストは天気の回答を返しません。提供元でエラーが起きた場合、エージェントは天気を作り上げず、失敗したことを説明する必要があります。

6. 演習2：ツールの契約と実行ループを読む

25-50分の後半 | ノートブック：コード確認と任意の強制なし実行セル

手順22. 表示されたソースを読む

ソース確認用セルを実行します。weather_contract.py と、短い起動用スクリプト 02_custom_tool_agent.py が表示されます。ツール名 get_current_weather と必須引数 city、country_code、units を探してください。

スキーマは受け付ける入力を定義します。実際の処理を担う実装はPython関数です。ツールの説明を渡すだけで、実行されるわけではありません。

手順23. 検証と呼び出しの箇所を見つける

execute_weather を探します。提供元に接続するアダプターの実行前に、引数が検証されることを確認してください。起動用スクリプトの execute 関数を見つけ、execute_weather を呼ぶ行を確認します。

units に指定できる値は何ですか。_____

未定義の引数が渡されたら、どう扱うべきですか。_____

手順24. 結果が戻る経路を確認する

ファイルパネルから agent_core.py を開きます。run_loop を探し、ツール結果メッセージの role: tool と tool_call_id を確認してください。この識別子により、結果と対応する要求が結び付きます。

ホストが会話ループを管理し、モデルとのやり取りを最大4ターン、ツール要求を最大4回に制限します。繰り返し呼び出しを止めるための制限であり、回答の正しさを保証するものではありません。

次の流れを自分の言葉で完成させてください。

質問+スキーマ-> _____

検証済みの引数-> _____

ツール結果+対応する呼び出しID-> _____

手順25. 任意：モデルに選択を任せる

OpenAIが動作し、時間が残っていれば、任意セルの RUN_UNFORCED = True を設定して実行します。最初のターンでツールを使う必須条件が外れます。モデルが実際にどう動くか観察してください。

モデルがツールを自発的に使用した：はい [] いいえ [] 未実施 []

台本に沿った代替モードでは、この実験は省略します。モデル自身の選択を観察することはできません。

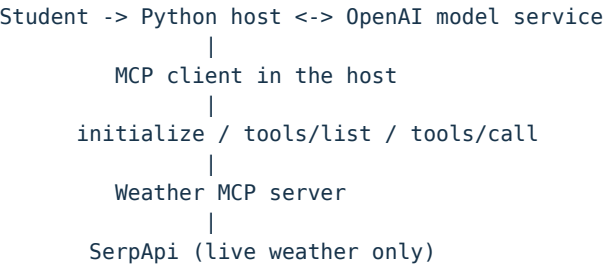
50-55分：休憩

再開する前に、次の文をペアの相手に説明してください。モデルが要求し、アプリケーションのコードが実行し、結果がモデルに戻ります。

7. MCPが受け持つ範囲を理解する

55-70分 | ノートブック：What MCP changes

手順26. 構成要素を整理する



構成要素	本講座での役割
ホスト (Host)	会話、モデルへのアクセス、検証、実行回数の上限を管理する
MCPクライアント	ホストとサーバーを接続し、プロトコルの要求を送る
MCPサーバー	天気ツールの情報を公開し、その実装を実行する
提供元のAPI	ライブモードで外部の天気検索データを提供する

手順27. 実行前のツール検出を追う

クライアントはセッションを初期化し、tools/list を要求します。サーバーはツール名、説明、入力スキーマを公開します。ホストはそのスキーマをモデル用のツール形式に変換します。その後、クライアントが要求された引数を指定して tools/call を送ります。

サーバーとクライアントは標準入出力、つまり stdio で通信します。両者は同じColabランタイム内のプロセスです。サーバーの標準出力はプロトコルメッセージ専用です。通常のデバッグ用print文をここへ出すと、通信を壊すおそれがあります。

手順28. 変わる点と変わらない点を説明する

演習2では、ホストがスキーマを定義して関数を直接呼び出しました。演習3では、ホストがスキーマを検出し、MCP経由で実行を要求します。天気アダプターはそのまま存在し、サーバーの内側で動きます。モデルとの会話は引き続きホストが管理します。

自分の言葉で： MCPは何を標準化しますか。 _____

Colabのlocalhostは、手元のPCではなくホスト型ランタイムを指します。OpenAIがこのstdioサーバーへ直接接続するわけではありません。LM Studioは、ホスト型の授業とは別に手元のPCで試す任意演習です。MCP自体は、外部データや生成文の正確性を保証しません。

8. 演習3：MCPツールを検出して呼び出す

70-100分 | ノートブック：Lab 3 - Real MCP discovery and execution

手順29. サーバーのコードを読む

Lab 3の最初のセルを実行します。weather_mcp_server.pyが表示されます。@mcp.tool()、型を指定した引数、execute_weatherの呼び出しを探してください。ここでサーバーが天気取得機能を公開しています。

手順30. 実際のプロトコル通信を確認する

同じセルで次の処理が実行されます。

```
run_lab("03_mcp_agent.py", "--inspect")
```

[MCP tools/list] を探し、get_current_weatherとその説明、パラメーターを確認します。サーバーの入力スキーマは、モデル用ツール形式へ変換された後、parametersとして表示されます。続いて [MCP tools/call] とサンプルの結果を探してください。

OpenAIやSerpApiの認証情報がなくても、これは実際のローカルMCP実行です。

手順31. モデルの実行ループにつなぐ

次のセルを実行します。

```
run_lab("03_mcp_agent.py", *model_options())
```

ツール検出、モデルへの要求、検証済み引数、MCP呼び出し、結果を順に追います。キーなしの経路では、モデルが模擬処理であることがログに明示されているか確認してください。

手順32. 責任範囲を比較する

確認事項	独自関数のエージェント	MCPエージェント
スキーマはどこから来るか	_____	_____
会話は誰が管理するか	_____	_____
天気コードはどのプロセスで動くか	_____	_____
ライブ天気用キーを使うのは誰か	_____	_____

手順33. 実行の証拠をペアで確認する

実際のツール呼び出しと返されたデータを示してください。ツール検出の結果は、ツール情報が公開された証拠です。それだけでは実行された証拠にはなりません。

ツール検出を確認 [] ツール結果を確認 []

実際のモデル推論を完了 [] モデル推論は未実施 []

9. ペア課題とエラーの切り分け

100-113分 | ノートブック：Pair challenge and failure exercise

手順34. 要求を変更する

OpenAIを使う場合は、ポルトガルのリスボンの気温を華氏で尋ねる課題セルを実行します。モデルが意図した都市、国、単位を指定したか確認してください。サンプル生成処理は、どの都市でも合成データを返します。華氏69.8度は、リスボンの実際の観測値ではありません。

推論を使わない場合、セルは直接サンプルの華氏変換を繰り返します。固定のフィクスチャと単位変換について、ペアの相手に説明してください。

手順35. データを照合する

要求した場所と単位： _____

返された場所と単位： _____

サンプルデータであると明記されていますか： _____

現在の天気と呼ぶ前に何を確認しますか： _____

手順36. 異常系テストを実行する

ノートブックのテスト用セルを実行します。独立した環境内でパッケージのテストが動きます。配布版には16件のテストがあります。最後のテスト要約に OK と表示されることを確認してください。実際の提供元の認証情報は使いません。

手順37. 2つの検証箇所を見つける

tests/test_training.py を開き、実際のMCP呼び出しで単位に invalid を指定するテストと、test_text_without_tool_withheld という名前のテストを探します。

異常の内容	どこで検出するか	何が分かるか
無効な気温の単位	_____	_____
ツールを使わない天気の記事	_____	_____

テスト成功で確認できるのは、そのテストが対象とする模擬処理やローカルのプロトコル動作です。OpenAI アカウントの利用可否や、ライブ天気の鮮度を証明するものではありません。

手順38. 1分で説明する

ホストが別のモデルサービスを使う場合、何が変わるかをペアの相手に説明してください。モデルへの接続設定を変えても、天気ツールとサーバーは同じものを使えます。このホスト型の授業は、手元のPCのLM Studio サーバーには接続しない点も伝えてください。

相手からのコメント： _____

10. 任意のライブデモと復旧手順

講師主導の追加演習です。予定済みの時間枠内、または授業後に実施します。

ライブ天気取得の手順（任意）

- 1. Colab Secretsに SERPAPI_KEY を追加し、ノートブックからのアクセスを有効にします。
- 2. Preparation Cを再実行して、ランタイムに認証情報を読み込ませます。
- 3. 任意のライブデモ用セルで RUN_LIVE_WEATHER を True に変更します。
- 4. セルを1回実行します。APIを直接呼び出し、OpenAIが有効ならMCPエージェントも実行します。
- 5. 場所、気温、単位、観測情報のラベル、情報源を確認します。デモ後は、意図しない繰り返し呼び出しを防ぐため、フラグを False に戻します。

ライブ要求は提供元の利用枠を消費し、OpenAI推論はAPI利用を伴います。検索結果に天気のanswer boxが含まれない場合もあります。推測値で補わず、取得できなかったことを説明してください。SerpApiはGoogleの天気検索結果を返します。元の情報提供者名は、結果内に明記されている場合のみ示します。取得時刻と観測時刻は同じではありません。

最初に失敗した箇所から切り分ける

症状	次に行うこと
LAB、run_lab、PYTHON が未定義	Preparation A、B、C、Dを順に実行する
パッケージの読み込み／導入に失敗	Bを再実行し、導入に失敗したら講師に相談する
キー追加後もOpenAIモードにならない	シークレットへのアクセスを有効にしてCを再実行する
モデルの認証／利用枠エラー	画面共有を避けてアカウントとキーを確認し、解決しなければ代替モードへ移る
モデルが見つからない	モデルIDとアクセス権を講師に確認してもらう
MCP起動／ツール検出のエラー	Dを再実行し、モデル推論の前に準備状態を確認する
利用できる天気結果がない	都市・国と返されたエラーを確認し、推測で補わない
ランタイムが置き換わった／切断された	再接続して準備をやり直し、保存済みノートブックからメモを復元する

キーなしの学習用経路へ切り替える

準備完了後、追加したセルで明示的に代替モードを使うには、次を実行します。

```
run_lab("02_custom_tool_agent.py", "--offline")
run_lab("03_mcp_agent.py", "--offline")
```

このコマンドは、外部モデルや天気サービスへリクエストを送りません。MCP処理は実際に実行されます。モデル推論は未実施として記録してください。授業中は復旧を1回試して解決しなければ、ペアで作業し、仕組みを理解する演習を最後まで進めましょう。

11. 理解度確認と次の実験

113-120分 | まず自分で記入し、その後で講師と確認します。

1. 構造化されたツール要求があれば、APIは実行済みといえますか。理由も説明してください。

2. MCPは何を標準化しますか。APIに関する処理で、引き続き必要なものは何ですか。

3. モデルのバックエンドを差し替えるとき、どの部分が変わりますか。

4. サンプル天気と実際のモデル推論を使う場合と、台本に沿った代替モードでは、何が違いますか。

5. 利用できる天気結果が得られなかったら、エージェントはどうすべきですか。

完了チェック

- ☐ 直接サンプルを実行し、各フィールドを確認した。
- ☐ 要求、検証、実行、結果、回答の流れを説明できる。
- ☐ 実際のMCPツール検出と実行を確認した。
- ☐ サンプル値とライブ観測値を区別できる。
- ☐ 実際のモデル推論を完了した、または未実施として記録した。
- ☐ コード、メモ、出力に認証情報を残さず、ノートブックを保存した。

次に試してみたい実験： _____

参考資料と本書の範囲

[Colab FAQ：ノートブック、共有、ランタイム](#) | [Colab Secrets API](#)

[OpenAIの関数呼び出し](#) | [MCP公式Python SDK v1](#)

講師用ガイドと解答例

本ドラフトは、リポジトリのColabノートブックに対応しています。14個のコードセルと16件のテストは、認証情報を使わずにローカルで確認済みです。ホスト型Colab、Secretsへのアクセス、OpenAI／SerpApiのライブ実行は、引き続き講師による事前リハーサルが必要です。本書は受講者用の演習ガイドであり、ライブサービスの利用可否を保証するものではありません。

著者：Rajkumar Rajagobalan